FIXORA — Vibe Coding Prompt Plan (SIH 2026 Prototype)

Team7 · PS 26261 · Built from your PPT and the four research reports

How to run this plan

1. Do the Stitch prompts first (separate doc). You need the exported UI before step P15.
2. Run **P0** once. It creates a `RULES.md` that every later prompt depends on, so the AI keeps the same names, statuses and API routes across the whole build.
3. Run P1 to P21 **in order**. Do not skip: each one assumes the previous one works.
4. After each prompt: run the tests, check its **Done when** line, commit to Git, then ask the AI: "List any deviation from RULES.md and the API contract." Fix deviations before moving on.
5. Use a coding agent that can run commands and edit many files (Claude Code, Cursor agent mode or similar). Keep each prompt in a fresh chat and tell the agent to read `RULES.md` first.

Stack (taken from your PPT slide 3)

Area	Use	Notes
Frontend	React + Tailwind CSS (Vite, TypeScript)	Charts with Chart.js; the PPT also lists Plotly and D3, optional
Backend	FastAPI (Python)	The PPT text says FastAPI, its icons show Django. This plan follows FastAPI
Databases	PostgreSQL + MongoDB	Postgres for relational and audit data, Mongo for OCR pages, extracted JSON and handbook chunks
AI/ML	Tesseract OCR, spaCy, HuggingFace + Sentence-BERT + FAISS, pandas, NumPy, scikit-learn	OpenCV and imagehash added for image checks

Area	Use	Notes
Deployment	Docker Compose, GitHub, GitHub Actions	Kubernetes and AWS are an optional last step

Left out on purpose: INGRES API, ARIMA, LSTM and gTTS. They come from a groundwater chatbot template and have no use in an application-validation system.

Prototype scope: one category (Extension of Approval), synthetic data only, mock AICTE records, mock issuer verification clearly labelled `SIMULATED SOURCE`. Checklist items and thresholds are placeholders until you copy the real values from the AICTE Approval Process Handbook.

Build order

Block	Prompts
Foundation	P0 rules file, P1 scaffold, P2 database, P3 auth, P4 audit
Core logic	P5 rule engine, P6 applications and uploads, P7 synthetic data, P8 OCR and extraction, P9 validation
Trust layer	P10 anomaly, P11 verification, P12 review workflow, infrastructure and alerts
Intelligence and reporting	P13 RAG chatbot, P14 analytics, report and evaluation
Frontend	P15 foundation, P16 institution, P17 officer, P18 admin and audit, P19 assistant, alerts and live progress
Finish	P20 end-to-end QA and demo, P21 deployment (optional)

P0 — Project rules file (run once)

```
Create a file RULES.md at the repo root with exactly the content below, then create CLAUDE.md or .cursorrules that says "Read RULES.md before every task." Do nothing else.

# FIXORA RULES
Product: AI-assisted application validation and document verification for AICTE Extension of Approval (EoA) applications. DECISION SUPPORT ONLY: the system flags issues with evidence; authorised humans decide. No endpoint or UI path may approve, reject or penalise an application automatically. Prototype uses synthetic data and mock integrations.

STACK (fixed): React + TypeScript + Vite + Tailwind CSS, Chart.js | FastAPI (Python 3.11), SQLAlchemy 2, Alembic, Pydantic v2 | PostgreSQL 16 + MongoDB 7 | Tesseract OCR (eng+hin), PyMuPDF, OpenCV, imagehash, spaCy, scikit-learn, pandas, NumPy, rapidfuzz, sentence-transformers (all-MiniLM-L6-v2) + FAISS | Docker Compose, GitHub Actions.

REPO LAYOUT:
backend/app/{core,models,schemas,routers,services,workers,adapters,rules}, backend/tests, frontend/src, data/{aph,synthetic,rulepacks}, infra/, storage/ (private, git-ignored).

CONVENTIONS: JSON snake_case. Errors: {"error": {"code":"...","message":"..."}}. Lists: {"items": [],"total":0,"page":1,"page_size":20}. Timestamps ISO-8601 UTC. IDs: APP-EOA-YYYY-NNNNN, F-YYYY-NNNNNN, D-NNN. Every module has pytest tests. Config via env vars only.

VOCABULARY (use these exact strings everywhere):
- Roles: institution, scrutiny_officer, verification_officer, admin
- Application states: DRAFT, SUBMITTED, VALIDATION_IN_PROGRESS, DEFICIENCIES_FOUND, RECTIFICATION_REQUESTED, VERIFICATION_PENDING, READY_FOR_SCRUTINY, UNDER_OFFICIAL_REVIEW, REFERRED_FOR_INSPECTION, CLOSED_BY_AUTHORITY. The system may move a case only among the automated states up to READY_FOR_SCRUTINY. UNDER_OFFICIAL_REVIEW and later need an authenticated officer action.
- Verification status: VERIFIED, MISMATCH_DETECTED, EXPIRED, VERIFICATION_PENDING, SOURCE_UNAVAILABLE, MANUAL_REVIEW_REQUIRED
- Checklist result: PRESENT, MISSING, EXPIRED, INCONSISTENT, VERIFIED, NEEDS_EVIDENCE
- Severity: info, needs_attention, review_required
```

```
- Finding type: missing, incomplete, expired, inconsistent, record_mismatch,
possible_anomaly, duplicate, verification, infrastructure
- Infrastructure output: consistent, needs_additional_evidence,
recommend_inspection (never pass/fail)
- Officer action: confirm, dismiss, request_rectification, escalate,
refer_inspection (a written reason is mandatory)

LANGUAGE: never write fake, forged or fraud in code strings, UI or reports.
Anomaly findings carry the label "Preliminary indicator - human verification
required". A source outage is never an adverse conclusion. Mock integrations
return simulated=true and the UI shows SIMULATED SOURCE.

FINDING SHAPE: {finding_id, application_id, type, severity, title, reason,
evidence:[{kind:region|record|metadata|source|rule, document_id, page, bbox,
field, value, measure, threshold, record_ref}], rule_refs:[], confidence,
label, status:open|confirmed|dismissed|rectification_requested|escalated,
model_versions:{}, created_at}. A finding without evidence cannot be saved.

VISIBILITY: institution users never see possible_anomaly or duplicate
findings, review priority scores or internal notes. They see deficiencies in
plain language.

API CONTRACT (all under /api):
- POST auth/login; GET auth/me
- POST applications; GET applications (role-scoped, filters:
status,state,category,severity,assignee,q,page); GET applications/{id}; POST
applications/{id}/submit; GET applications/{id}/checklist; POST
applications/{id}/validate; GET applications/{id}/status (pipeline stage
progress); GET applications/{id}/findings; GET applications/{id}/report.pdf
- POST applications/{id}/documents (multipart); GET documents/{id}; GET
documents/{id}/file (secure stream); GET documents/{id}/fields; GET
documents/{id}/ocr; POST documents/{id}/verify
- GET findings/{id}; POST findings/{id}/actions
- GET verification/queue; POST verification/{id}/manual-confirm; GET
inspection/queue
- GET/POST rule-packs; GET rule-packs/{id}; POST rule-packs/{id}/submit; POST
rule-packs/{id}/approve; POST rule-packs/{id}/test
- POST chat; GET alerts; POST alerts/{id}/ack; GET analytics/overview
- GET audit; GET audit/verify-chain
- POST demo/seed; POST demo/reset
```

```
PRINCIPLES: rules are data (YAML), not code. Deterministic rules decide
requirements; ML only extracts and flags. Low confidence routes to manual
review. Original uploads are immutable. Every state change, rule run, source
call, officer action and document read writes an audit event.
```

Done when: `RULES.md` exists and the agent config points to it.

P1 — Scaffold and Docker Compose

```
Read RULES.md. Create the monorepo layout from RULES.md. Backend: FastAPI app
with a /api/health route that checks PostgreSQL and MongoDB connectivity,
CORS for the frontend, structured logging, settings from env vars, pytest set
up. Frontend: Vite + React + TypeScript + Tailwind, a placeholder page that
calls /api/health and shows the result. docker-compose.yml with postgres:16,
mongo:7, backend (hot reload), frontend (hot reload), named volumes,
healthchecks. Add .env.example, .gitignore (storage/, .env), a Makefile (up,
down, test, seed, eval, lint), a README with run steps, and a GitHub Actions
workflow that runs backend lint and tests. Install Tesseract with eng and hin
language data in the backend Dockerfile.
```

Done when: `docker compose up` starts all four services, `/api/health` reports both databases ok, the frontend shows it, CI passes.

P2 — Database models and migrations

```
Read RULES.md. Implement PostgreSQL models with SQLAlchemy 2 and Alembic migrations: users, institutions, applications (category, academic_year, checklist_pack_version, state, priority_score, assigned_to, submitted_at, form_data JSONB), documents (class, file_ref, sha256, phash, page_count, version, scan_status, classification_confidence), extracted_fields (document_id, name, raw_value, normalized_value, page, bbox, confidence, model_version), checklist_results (application_id, rule_id, rule_version, result, inputs_ref), findings, evidence, verification_records (immutable, new attempt = new row), officer_actions, alerts, audit_entries, rule_packs, rule_pack_versions, ref_records (mock AICTE records: previous_eoa, approved_programmes_and_intake, faculty_count, ter_payment_status, deficiency_status, per institution), pipeline_jobs. Use the enums from RULES.md. In MongoDB create collections with indexes: ocr_pages {document_id, page, text, words[{text,bbox,conf}], engine}, extraction_runs, aph_chunks {section, page, text, embedding_id}. Add repository helpers and tests for constraints.
```

Done when: `alembic upgrade head` creates every table, Mongo indexes exist, model tests pass.

P3 — Authentication and role-based access

```
Read RULES.md. Implement JWT authentication (bcrypt, access token 60 minutes), POST /api/auth/login and GET /api/auth/me, a require_roles dependency, and institution scoping so an institution user can only read its own applications and documents. Add per-field masking helpers for personal data in responses and logs. Add basic rate limiting on login. Seed demo users: one per role plus a second institution user for a different institution, all with documented demo passwords in README. Return 403 with the standard error shape for forbidden access. Tests: every role against every protected route group, cross-institution access denied.
```

Done when: login works for all demo users and the access tests pass.

P4 — Audit service with hash chain

```
Read RULES.md. Build an audit service: log_event(actor, role, action,
target_type, target_id, payload, rule_versions, model_versions) writes an
append-only audit_entries row with seq, payload_hash and prev_hash (SHA-256
chain). Prevent UPDATE and DELETE with a PostgreSQL trigger. Add middleware
or a dependency that logs login, document reads and officer actions.
Implement GET /api/audit (filters: actor, role, event, application, date
range, pagination; visible to scrutiny_officer and admin) and GET
/api/audit/verify-chain that re-computes the chain and returns {valid,
checked, first_broken_seq}. Tests: normal chain is valid; a tampered row
makes verify-chain fail at the right sequence number.
```

Done when: chain verification passes, and fails in the tamper test.

P5 — Rule engine and rule packs

```
Read RULES.md. Build the rules module. A rule pack is YAML: id, category,
handbook_edition, version, effective_from, items[]. Supported item types:
document_required, document_validity (max_age_days or must_not_be_expired),
field_required, field_format (regex), cross_field_consistency,
form_to_document_match, form_to_record_match (with tolerance), record_status,
numeric_threshold, conditional (if/then), infrastructure_norm. Each item has
id, clause_ref, severity and a message template. Write a pure, deterministic
evaluator evaluate(pack, case_context) that returns results with rule_id,
rule_version, result (checklist vocabulary), inputs and a plain-language
message. Create the seeded pack EOA-CHECKLIST v1 with about 15 items
(affidavit, fire safety certificate, building or occupancy certificate,
accreditation certificate, TER payment, land document, intake match, name
match, faculty count, lab count, conditional NOC) and mark every threshold
and handbook value as PLACEHOLDER. Store packs in rule_packs and
rule_pack_versions with maker-checker status Draft, In review, Approved,
Retired (the approver must differ from the author; new versions apply only to
new applications). Endpoints from RULES.md for rule-packs. Unit tests: every
rule type with passing and failing cases, boundary dates, version pinning.
```

Done when: `pytest backend/tests/rules` passes, and the same input plus the same version always gives the same output.

P6 — Applications and secure document upload

```
Read RULES.md. Implement application endpoints (create, list with role scoping and filters, detail, submit) and document upload: validate extension, MIME and magic bytes, size and page limits; compute SHA-256; run a virus-scan stub behind an interface; encrypt files at rest with Fernet in storage/ with random names; keep every version (replacement creates version+1, the original is never overwritten); never expose file paths. GET /api/documents/{id}/file streams a decrypted copy to authorised users only and writes an audit event. Enforce the state machine from RULES.md with explicit allowed transitions and tests for illegal ones. The create-application form accepts institution_id, category, academic_year, change_type and programmes[{name,intake}] in form_data. Emit audit events for every change.
```

Done when: an institution user can create an application, upload and replace documents, and another institution cannot read them.

P7 — Synthetic data generator

```
Read RULES.md. Write data/synthetic/generate.py (deterministic with a seed, runnable via make seed and POST /api/demo/seed). Create 6 institutions with ref_records, and 40 applications. For each application generate 6 to 9 documents as PDF or PNG using reportlab and Pillow: affidavit, fire safety certificate, building or occupancy certificate, accreditation certificate (with a QR code), TER payment receipt, land document, floor plan (labelled rooms, labs), and 2 to 3 infrastructure photographs (placeholder images with EXIF). Inject known defects with a ground_truth.json (application_id, document_id, defect_type, expected_detector): missing document, expired certificate, swapped institution name, intake mismatch against ref_records, altered date (pixel edit then re-save), reused certificate or photo across institutions, blurred or cropped scan, QR payload mismatch, source unavailable, floor plan with fewer labs than declared. Also create five named demo applications (clean, missing and expired, mismatch, altered date and duplicate, infrastructure shortfall). Label all data SYNTHETIC in document footers. Add POST /api/demo/reset.
```

Done when: `make seed` creates the 40 applications and documents, `ground_truth.json` exists, and the five named demo cases appear in the database.

P8 — OCR, classification and extraction pipeline

```
Read RULES.md. Build a background pipeline using FastAPI BackgroundTasks plus the pipeline_jobs table (stages: UPLOADED, CLASSIFIED, EXTRACTED, FAILED) so progress can be polled. Steps per document: file triage; preprocessing (deskew, denoise, contrast with OpenCV); PyMuPDF text layer first, Tesseract (eng+hin) as fallback; page text and word boxes stored in Mongo ocr_pages; document classification with a scikit-learn TF-IDF + logistic regression model trained on the synthetic documents, with rule-based keyword fallback and a confidence value, unknown or low confidence goes to MANUAL_REVIEW_REQUIRED; field extraction per document class using regex and spaCy entities (institution_name, registration or certificate number, issue date, expiry date, signatory, programme, intake, area, lab count, authority, QR payload), normalisation of dates, names and units while keeping raw values; every field saved with page, bbox, confidence and model_version. Endpoints: GET documents/{id}/fields, GET documents/{id}/ocr, GET applications/{id}/status. Add backend/app/eval/extraction_eval.py that compares output to ground truth and prints per-field accuracy.
```

Done when: every seeded document is classified and extracted, and the eval script prints accuracy per field and per document class.

P9 — Validation engine and findings

```
Read RULES.md. Implement POST /api/applications/{id}/validate (idempotent, runs as a background job). It loads the application's pinned rule pack, builds the case context from form_data, extracted fields and ref_records, runs the evaluator, adds fuzzy cross-document and form-to-document matching with rapidfuzz using field-specific thresholds (a fuzzy match is a flag, never an identity conclusion), then writes checklist_results, findings and evidence rows (page and bbox for every document-based finding, record_ref for ref_records). Generate plain-language explanations from templates, not free text. Compute a 0 to 100 review priority from severity, number of corroborating signals and deadline proximity (never shown to institutions). Supersede old findings on a re-run instead of deleting them. Move the application state automatically (DEFICIENCIES_FOUND or READY_FOR_SCRUTINY or VERIFICATION_PENDING). Create alerts for missing and expiring documents. Audit every step with rule and model versions. Endpoints: GET applications/{id}/checklist and /findings with role-based filtering. Tests against ground truth for missing, expired, name swap and intake mismatch.
```

Done when: running validation on the demo applications produces the expected findings, and the institution token cannot see internal fields.

P10 — Anomaly and potential-alteration module

```
Read RULES.md. Add an anomaly service called from the validation job. Checks: (1) metadata (PDF and image producer, creation and modification times, incremental updates, modification after the stated issue date); (2) Error Level Analysis and noise or compression inconsistency with OpenCV, localised to regions and saved as bbox with measure and threshold; (3) date and identifier logic (issue after expiry, issue after submission, certificate number format); (4) duplicate and reuse detection with imagehash perceptual hashes across all applications, plus near-duplicate text using Sentence-BERT embeddings in FAISS; (5) quality checks for blur or crop. An aggregator combines signals so one weak signal alone never creates a finding. Every result is saved as possible_anomaly or duplicate with evidence, confidence and the mandatory label. Never emit forged, fake or fraud. Write backend/app/eval/anomaly_eval.py that reports precision, recall and false positives per 100 documents against ground truth, per defect type.
```

Done when: the altered-date and reused-certificate demo cases are flagged with highlighted regions, and the eval prints precision and recall.